Web scraping of Wiki page

Valentina Andreeva

2026-10-26

Contents

Web Scraping 1

Research question and variables . . . 2
Data . . . 2
Web scraping from the main page . . . 6
Web scraping from individual pages . . . 7
Joining tables . . . 10
EDA . . . 11
Cleaning the dataset . . . 11
Validation checks . . . 13
Distribution of categorical variables . . . 14
Distribution of numeric variables . . . 15
Final dataset . . . 16
Visualization . . . 17

List of references 18

Date of data collection for web scraping: 2026-10-24.

Web Scraping

In this section, we will extract a table from the Wikipedia page listing actors (Wikipedia, 2026), as well as some data from these actors' personal pages, in order to determine which factors are associated with whether a person has won an Oscar.

Research question and variables

RQ: What factors are associated with whether a person has won an Oscar?

Variables we plan to extract **from the main page**, which we will need later for logistic regression:

- `sex` (binary categorical) - sex of a person, *control variable*
- `age` (numeric) - age of a person, *control variable*
- `born` (numeric) - year a person was born, *predictor*
- `address` (binary categorical) - place of induction: Vine Street or Hollywood Boulevard, *predictor*
- `age_at_star` (numeric) - age a person was inducted, *predictor*
- `oscar_won` (binary categorical) - whether a person won Oscar or no, *target variable*

Variables from the actors' personal pages, which we will also include in the logistic regression as predictors:

- `n_children` (numeric) - number of children, *predictor*
- `birth_country` (categorical) - country of origin, *predictor*

It is worth noting in advance that variables related to the year may be correlated with one another and may need to be removed later in the analysis process. However, for now we will leave them in and remove them only if they fail the assumptions check prior to the analysis.

One row in our dataset would be one actor.

Data

Data collection date: October 24, 2026

Source: Wikipedia, 2026 (https://en.wikipedia.org/wiki/List_of_actors_with_Hollywood_Walk_of_Fame_motion_picture_stars)

We downloaded the pages for all the actors in advance using `01_collect.R` file and saved them in a separate folder to avoid any page changes that could disrupt the web scraping process. Consequently, we no longer need "polite" parsing, since in this file we do not interact directly with the website: we handled that in advance in a separate file. Presumably, this should reduce potential issues with the site blocking us or data changes, and, as a result, improve the reproducibility of the results. The code for downloading pages

followed “polite” rules (contact email, interval between requests, etc.); robots.txt allows parsing and the terms do not forbid it. See `01_collect.R` for more details:

```
# ============================================================
# Title:  Downloading html pages
# Author: Valentina Andreeva
# Date:   2026-10-24
# ============================================================
#
# Source URL:
 ↪  https://en.wikipedia.org/wiki/List_of_actors_with_Hollywood_Walk_of_Fame_motion_picture_st
#
# Date of collection: October 24, 2026.
#
# Robots.txt and terms were checked, web scraping is allowed.
# ============================================================

# ---- Packages ----
packages <- c("tidyverse", "rvest", "httr", "here", "stringr")
# missing <- packages[!(packages %in% installed.packages()[, "Package"])]
# if (length(missing) > 0) install.packages(missing)
invisible(lapply(packages, library, character.only = TRUE))

# Code to avoid problems with PC username on cyrillics
# ascii_tmp <- "C:/Rtmp"
# dir.create(ascii_tmp, showWarnings = FALSE, recursive = TRUE)
# Sys.setenv(TMPDIR = ascii_tmp, TMP = ascii_tmp, TEMP = ascii_tmp)

# This file is additional and its purpose is to automatically download the
 ↪  required
# pages for further web scraping. Since there are about 950 of them, downloading
# them automatically will be much more convenient and efficient.

# ---- Constants ----
# First, let's state the parsing settings in accordance with the rules of
 ↪  politeness:
# we will set a user agent with a contact email address and a 2-second interval
# between requests so that we do not overload the site. Moreover, robots.txt
 ↪  allows
# parsing and the terms do not forbid it.

MY_UA   <- "HSE-SPb Advanced Data Analysis course, assignment (contact:
 ↪  vaalandreeva@edu.hse.ru)"
```

```r
DELAY_S <- 2
OUT_DIR <- "data/raw"
TIMEOUT <- 60
TRIES   <- 3

# Now let's set a timeout for httr and create a folder where we will save the
 ↪  pages.

httr::set_config(httr::timeout(TIMEOUT))
dir.create(OUT_DIR, showWarnings = FALSE)

# ---- Parsing from the main page ----
# Let's specify the file from which we will extract pages, and write a function
 ↪  to do so.
# We need to retrieve links to actors from the second table on the main actors
 ↪  page.

main_file <- "data/raw/main_list.html"

main_page <- read_html(main_file)
all_tables <- main_page %>% html_elements("table")

tbl_node <- all_tables[[2]]

# Now let's extract the links to the actors' pages from the main page.

urls <- tbl_node %>% html_elements("tr") %>% .[-1] %>%
  map_chr(\(row) {
    a <- row %>% html_element("td:nth-child(1) a")
    if (length(a) == 0) NA_character_ else html_attr(a, "href")
  }) %>%
  na.omit() %>%
  url_absolute(base = "https://en.wikipedia.org")

# Let's extract the article titles from the links so we can name the files after
 ↪  the articles.

titles <- urls %>%
  str_remove("^https://en\\.wikipedia\\.org/wiki/") %>%
  URLdecode()
```

```r
safe_title <- titles %>%
  str_replace_all("[^A-Za-z0-9_\\-]+", "_") %>%
  str_replace_all("_+", "_") %>%
  str_remove("^_|_$") %>%
  str_sub(1, 80)

files <- file.path(OUT_DIR,
                   sprintf("%03d_%s.html", seq_along(urls), safe_title))

# ---- Working with the actors' pages ----
# Now let's create a function to download the pages themselves.
# To make the download process more efficient, we have added a feature
# that prevents already downloaded articles from being downloaded again.

already_ok <- file.exists(files) & file.size(files) > 5000

download_one <- function(url, file) {
  if (file.exists(file) && file.size(file) > 5000) return("already")
  for (i in seq_len(TRIES)) {
    Sys.sleep(DELAY_S)
    resp <- tryCatch(
      httr::GET(url, httr::user_agent(MY_UA), httr::timeout(TIMEOUT)),
      error = function(e) NULL
    )
    if (!is.null(resp) && !httr::http_error(resp)) {
      txt <- httr::content(resp, "text", encoding = "UTF-8")
      if (nchar(txt) > 5000) {
        writeLines(txt, con = file, useBytes = TRUE)
        return("ok")
      }
    }
    if (i < TRIES) Sys.sleep(5 * i)
  }
  "failed"
}

results <- character(length(urls))
t0 <- Sys.time()

# ---- Downloading actors' articles ----
# And finally, let's download our articles.
```

```
for (i in seq_along(urls)) {
  if (i %% 10 == 0 || i == 1) {
    done   <- sum(results[seq_len(i - 1)] %in% c("ok", "already"))
    elapsed <- as.numeric(difftime(Sys.time(), t0, units = "mins"))
    eta     <- if (i > 1) elapsed / (i - 1) * (length(urls) - i + 1) else NA
    cat(sprintf("[%4d / %4d] saved=%4d  elapsed=%.1f min  ETA=%.1f min\n",
                i, length(urls), done, elapsed, eta))
  }
  results[i] <- download_one(urls[i], files[i])
}
```

Web scraping from the main page

So, we will start parsing the downloaded pages rather than the website itself. First, we will read the main page and extract the table from it, adding NA if any data is missing.

```
main_file <- "data/raw/main_list.html"

main_page <- read_html(main_file, encoding = "UTF-8")
all_tables <- main_page %>% html_elements("table")
tbl_node <- all_tables[[2]]

actors_raw <- tbl_node %>%
  html_table(header = TRUE, fill = TRUE) %>%
  clean_names()

actor_links <- tbl_node %>%
  html_elements("tr") %>%
  .[-1] %>%
  map_chr(\(row) {
    a <- row %>% html_element("td:nth-child(1) a")
    if (length(a) == 0) NA_character_ else html_attr(a, "href")
  }) %>%
  url_absolute(base = "https://en.wikipedia.org")

head(actors_raw)
```

```
## # A tibble: 6 x 9
##   actor           x      born died    age address inducted at_age oscar
```

```
##   <chr>           <chr> <int> <chr> <int>   <int>    <int> <chr>  <chr>
## 1 Bud Abbott      M      1897 1974     76    1611     1960 63     ~
## 2 Art Acord       M      1890 1931     40    1709     1960 ~      ~
## 3 Amy Adams       F      1974 ~        52    6280     2017 43     Nom
## 4 Renee Adoree    F      1898 1933     35    1601     1960 ~      ~
## 5 Philip Ahn      M      1905 1978     72    6211     1984 ~      ~
## 6 Frank Albertson M      1909 1964     55    6754     1960 51     ~
```

Now let's link the article titles to their respective articles by adding a `Url` column to the table.

```
actors_raw <- actors_raw %>%
  mutate(
    url    = actor_links,
    title  = url %>%
      str_remove("^https://en\\.wikipedia\\.org/wiki/") %>%
      URLdecode()
  ) %>%
  rename(sex = x) %>%
  filter(!is.na(url))
```

Now let's check the files in the htmls folder and add the path to each of the corresponding files to the dataframe in `actor_file` column to make further parsing easier.

```
htmls_dir <- "data/raw/"
existing_files <- list.files(htmls_dir)

existing_files <- list.files(htmls_dir, pattern = "\\.html$", full.names = FALSE)
 ↪  %>%
  str_subset("^\\d{3}_")

actors_raw <- actors_raw %>%
  mutate(actor_file = file.path(htmls_dir, existing_files))
```

Web scraping from individual pages

Now let's write a function to parse the data from our list of websites. What we need to do is extract the last part of the .birthplace field to determine the country of birth, and also extract the number of names or a number from the section about children.

```
parsing_country <- function(text) {
  parts <- str_split(text, ",\\s*")[[1]] %>% str_trim() %>% .[. != ""]
  if (length(parts) > 0) parts[length(parts)] else NA_character_
}

get_td <- function(ths, label) {
  idx <- which(str_detect(ths %>% html_text2(), regex(label, ignore_case =
 ↪  TRUE)))
  if (length(idx) == 0) return(NULL)
  td <- ths[idx[1]] %>% html_element(xpath = "..") %>% html_element("td")
  if (length(td) == 0) return(NULL)
  td
}

parse_actor <- function(page, url) {
  infobox <- page %>% html_element("table.infobox")
  if (length(infobox) == 0)
    return(tibble(url = url, birth_country = NA_character_, n_children =
     ↪  NA_real_))

  ths <- infobox %>% html_elements("th")

  birth_country <- NA_character_
  bp <- infobox %>% html_element(".birthplace, .birth-place")
  if (length(bp) > 0)
    birth_country <- parsing_country(bp %>% html_text2() %>% str_squish())

  if (is.na(birth_country)) {
    td <- get_td(ths, "^Born$")
    if (!is.null(td)) {
      lines <- td %>% html_text2() %>% str_squish() %>%
        str_split("\\n") %>% .[[1]] %>% str_trim()
      lines <- lines[lines != "" & !str_detect(lines, "\\d{4}") &
                       !str_detect(lines, "^\\(.*\\)$")]
      if (length(lines) > 0) birth_country <- parsing_country(tail(lines, 1))
    }
  }

  n_children <- NA_real_
  td <- get_td(ths, "^Children$")
  if (!is.null(td)) {
```

```
    txt <- td %>% html_text2() %>% str_squish()
    num <- str_extract(txt, "\\d+")
    n_children <- if (!is.na(num)) as.numeric(num) else {
      parts <- str_split(txt, ",\\s*|\\s+and\\s+")[[1]]
      parts <- parts[parts != ""]
      if (length(parts) > 0) length(parts) else NA_real_
    }
  }

  tibble(url = url, birth_country = birth_country, n_children = n_children)
}
```

Now let's parse the individual actor pages to extract their country of origin and the number of children.

```
matched <- which(!is.na(actors_raw$actor_file))

actor_details <- map2(
  actors_raw$url[matched],
  actors_raw$actor_file[matched],
  \(url, file) {
    page <- tryCatch(read_html(file), error = function(e) NULL)
    if (is.null(page)) return(NULL)
    tryCatch(parse_actor(page, url), error = function(e) {
      message("Parse error ", basename(file), ": ", conditionMessage(e))
      NULL
    })
  },
  .progress = TRUE
) %>% compact() %>% list_rbind()
```

Let's take a look at the dataframe with details about the actors:

```
head(actor_details)
```

```
## # A tibble: 6 x 3
##   url                                          birth_country n_children
##   <chr>                                        <chr>              <dbl>
## 1 https://en.wikipedia.org/wiki/Bud_Abbott     U.S.                   2
```

```
## 2 https://en.wikipedia.org/wiki/Art_Acord       U.S.                  NA
## 3 https://en.wikipedia.org/wiki/Amy_Adams       Italy                  1
## 4 https://en.wikipedia.org/wiki/Renee_Adoree    France                NA
## 5 https://en.wikipedia.org/wiki/Philip_Ahn      U.S.                  NA
## 6 https://en.wikipedia.org/wiki/Frank_Albertson U.S.                   5
```

Joining tables

Finally, we will combine the data from the table on the main actors page with the data we additionally parsed from the individual actors' pages (number of children and country of origin) into a single table.

```
actors_full <- actors_raw %>%
  select(-actor_file) %>%
  left_join(actor_details, by = "url")
```

Let's take a look at the raw, complete table we ended up with:

```
head(actors_full)
```

```
## # A tibble: 6 x 13
##   actor        sex    born died    age address inducted at_age oscar url
 ↪  title
##   <chr>        <chr> <int> <chr> <int>   <int>    <int> <chr>  <chr> <chr>
 ↪  <chr>
## 1 Bud Abbott   M      1897 1974     76    1611     1960 63     ~     http~
 ↪  Bud_~
## 2 Art Acord    M      1890 1931     40    1709     1960 ~      ~     http~
 ↪  Art_~
## 3 Amy Adams    F      1974 ~        52    6280     2017 43     Nom   http~
 ↪  Amy_~
## 4 Renee Adoree F      1898 1933     35    1601     1960 ~      ~     http~
 ↪  Rene~
## 5 Philip Ahn   M      1905 1978     72    6211     1984 ~      ~     http~
 ↪  Phil~
## 6 Frank Alber~ M      1909 1964     55    6754     1960 51     ~     http~
 ↪  Fran~
## # i 2 more variables: birth_country <chr>, n_children <dbl>
```

EDA

Cleaning the dataset

We can see that the table needs to be cleaned.

First, let's replace the '~' with NA in the table from the main actors page, since that is the value implied by the tilda. Then:

- `oscar_won`: since we want to perform logistic regression with the target variable `oscar_won`, we need two categories instead of three (where nominated will be treated as "not won"). The threshold is: "Won" - 1; "Nom" or "~" - 0; any other value - NA. Nominations are collapsed into "not won" because a nomination is not a win, and keeping three categories would make the target non-binary.
- `birth_year`, `inducted_year`, `age_at_star`, `age`: we will create variables with descriptive names (`birth_year`, `inducted_year`, `age_at_star`) from the existing variables in the dataset, and change their data types to integer, as well as for the `age` variable.
- `birth_country`: we need to process the `birth_country` data and extract the country's name without any extra characters. Let's take the four English-speaking countries that appear most frequently in our dataset: United States, United Kingdom, Canada, and Australia. We will categorize the remaining countries under Other.
- `n_children`: we will also create an integer variable for the number of children `n_children` and remove outlier values that are likely inaccurate (greater than 50).
- `address`: finally, we will recode the street of induction codes into the actual `address`.

```
actors_clean <- actors_full %>%
  mutate(
    across(
      any_of(c("sex", "born", "age", "address", "inducted", "at_age")),
      ~ if_else(str_detect(as.character(.x), "^\\s*~\\s*$"),
                NA_character_, as.character(.x))
    )) %>%

  mutate(
    oscar_won = case_when(
      str_detect(oscar, regex("^Won$", ignore_case = TRUE))       ~ "1",
      str_detect(oscar, regex("^(Nom|~)$", ignore_case = TRUE))   ~ "0",
      TRUE ~ NA_character_),
```

```
    birth_year    = as.integer(born),
    age = as.integer(age),
    inducted_year = as.integer(inducted),
    age_at_star   = suppressWarnings(as.integer(at_age)),

    birth_country = case_when(
      is.na(birth_country) ~ NA_character_,
      str_detect(birth_country, regex("U\\.S\\.|USA|United States", ignore_case =
      ↪  TRUE)) ~ "United States",
      str_detect(birth_country, regex("UK|United Kingdom|England|Scotland|Wales",
      ↪  ignore_case = TRUE)) ~ "United Kingdom",
      str_detect(birth_country, regex("Canada",    ignore_case = TRUE)) ~
      ↪  "Canada",
      str_detect(birth_country, regex("Australia", ignore_case = TRUE)) ~
      ↪  "Australia",
      TRUE ~ "Other"
    ),
    birth_country = factor(birth_country),

    n_children = if_else(as.integer(n_children) > 50, NA_integer_,
    ↪  as.integer(n_children)),

    address_num = as.integer(str_extract(address, "^\\s*(\\d+)", group = 1)),
    address = case_when(
      address_num >= 6000 & address_num < 8000 ~ "Hollywood Boulevard",
      address_num >= 1000 & address_num < 2000 ~ "Vine Street",
      TRUE ~ NA_character_
    )) %>%
  select(c(-address_num, -born, -died, -inducted, -at_age, -oscar, -url, -title))
  ↪  %>%
  distinct()
```

Let's take a look at the cleaned dataset:

```
head(actors_clean)
```

```
## # A tibble: 6 x 10
##   actor        sex     age address birth_country n_children oscar_won
 ↪  birth_year
##   <chr>        <chr> <int> <chr>   <fct>              <int> <chr>
 ↪  <int>
```

```
## 1 Bud Abbott   M        76 Vine S~ United States          2 0
 ↪  1897
## 2 Art Acord    M        40 Vine S~ United States         NA 0
 ↪  1890
## 3 Amy Adams    F        52 Hollyw~ Other                  1 0
 ↪  1974
## 4 Renee Adoree F        35 Vine S~ Other                 NA 0
 ↪  1898
## 5 Philip Ahn   M        72 Hollyw~ United States         NA 0
 ↪  1905
## 6 Frank Alber~ M        55 Hollyw~ United States          5 0
 ↪  1909
## # i 2 more variables: inducted_year <int>, age_at_star <int>
```

Validation checks

First, let's check whether there are any **NA values** in the target variable, and also see which variables have the most NA values:

```
print(colSums(is.na(actors_clean)))
```

```
##         actor           sex           age       address birth_country
##             0             0             0             0             7
##    n_children     oscar_won    birth_year inducted_year   age_at_star
##           282             0             0             0           187
```

There are no NA values in the target variable, which means the dataset is ready for logistic regression. We also see that even in the variable with the highest number of missing values, about 25% of the observations are missing, which is not a lot, since there are 944 rows in the dataset.

Let's also take a look at the final **number of rows and columns**:

```
nrow(actors_clean); ncol(actors_clean)
```

```
## [1] 944
```

```
## [1] 10
```

Thus, we have 944 rows (actors) and 10 columns, in which the first one is a name of an actor, and the remaining ones contain the target and 8 predictors.

Let's check for **duplicates** in the original and cleaned data:

```
sum(duplicated(actors_full))
```

```
## [1] 6
```

```
sum(duplicated(actors_clean))
```

```
## [1] 0
```

The first file contained 6 duplicates, and we removed them. They appeared because three actors shared a single Wikipedia page, and the rows containing them were repeated three times.

Let's also take a look at the **ratio of those who won an Oscar** to those who did not.

```
print(table(actors_clean$oscar_won))
```

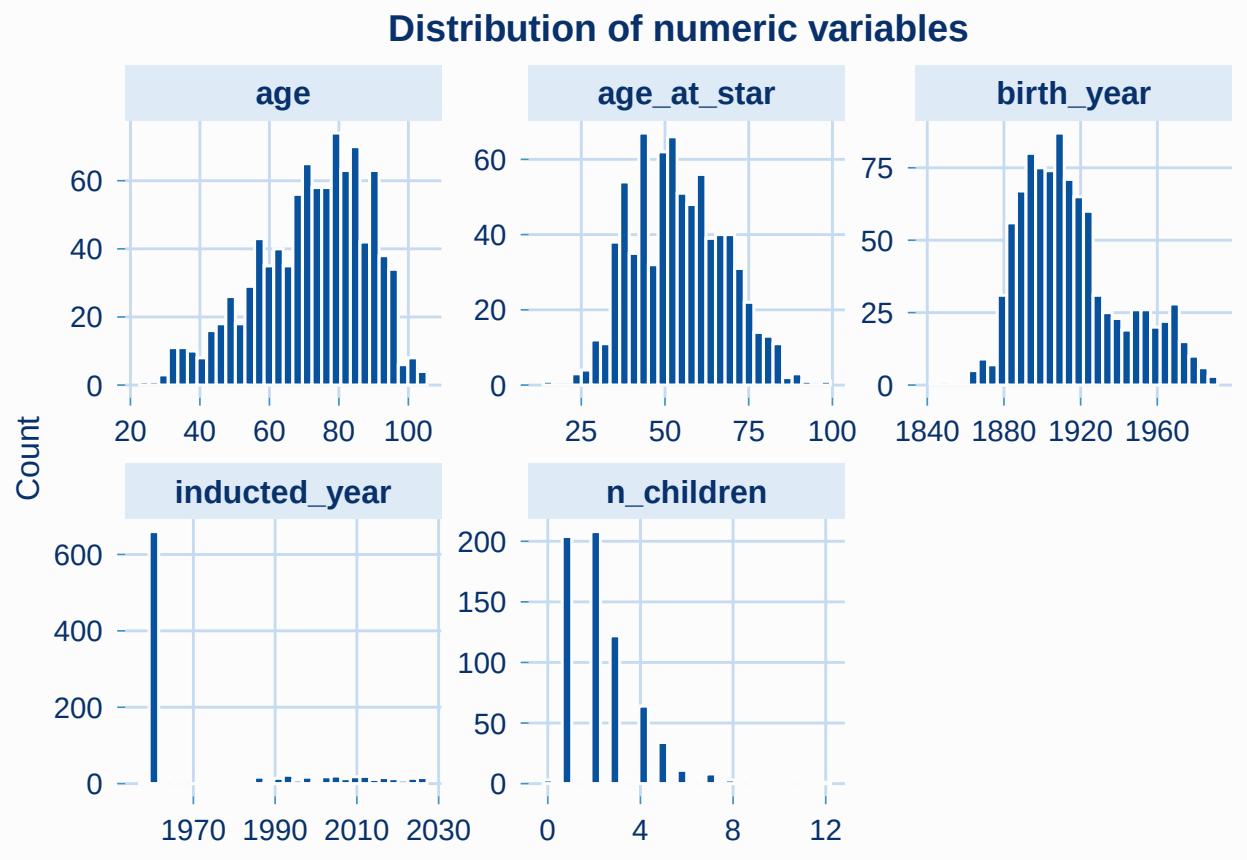
```
##
##   0   1
## 761 183
```

We can see that the least popular category of the target variable accounts for approximately 20% of the data, which is enough for logistic regression.

Distribution of categorical variables

Let's now look at the distribution of each categorical variable:

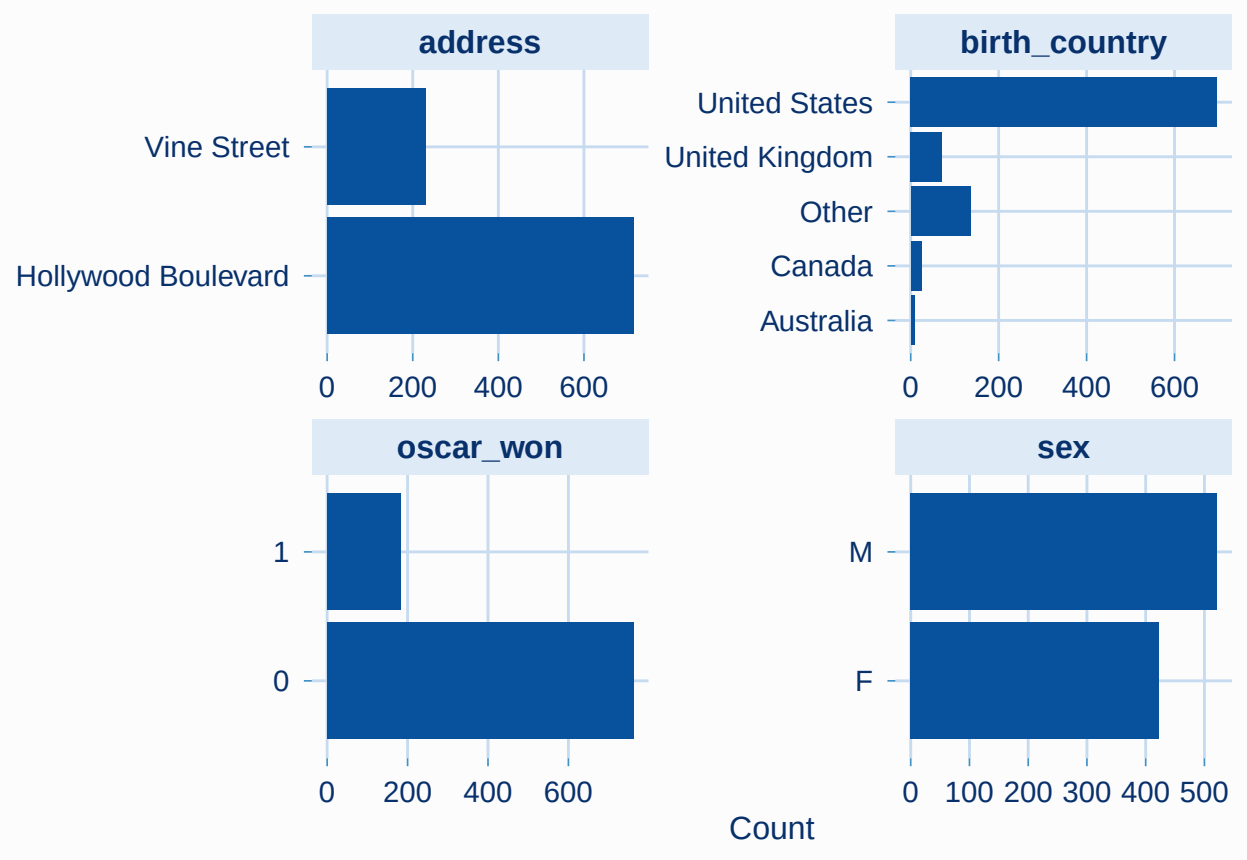
```
actors_clean %>%
  select(where(is.numeric)) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = '#08519C', color = "#FCFCFC") +
  facet_wrap(~ variable, scales = "free") +
  labs(title = "Distribution of numeric variables",
       x = NULL, y = "Count") +
  theme_myreport()
```



The distributions of the variables `age`, `age_at_star`, and `birth_year` are approximately normal. However, the distributions of `inducted_year` and `n_children` are highly non-normal and right-skewed.

Distribution of numeric variables

```
actors_clean %>%
  select(where(~ is.character(.x) || is.factor(.x)), -any_of("actor")) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  filter(!(variable == "birth_country" & is.na(value))) %>%
  count(variable, value) %>%
  ggplot(aes(x = value, y = n)) +
  geom_col(fill = '#08519C') +
  facet_wrap(~ variable, scales = "free") +
  coord_flip() +
  labs(x = NULL, y = "Count") +
  theme_myreport()
```



We can see that the distribution is uneven across all categorical variables except for `sex`; however, in all binary variables, each category accounts for at least 10% of the data.

Final dataset

```
glimpse(actors_clean)
```

```
## Rows: 944
## Columns: 10
## $ actor         <chr> "Bud Abbott", "Art Acord", "Amy Adams", "Renee Adoree",
 ↪  ~
## $ sex           <chr> "M", "M", "F", "F", "M", "M", "M", "M", "M", "F", "F",
 ↪  "~
## $ age           <int> 76, 40, 52, 35, 72, 55, 58, 78, 73, 71, 88, 62, 39, 88,
 ↪  ~
## $ address       <chr> "Vine Street", "Vine Street", "Hollywood Boulevard",
 ↪  "Vi~
```

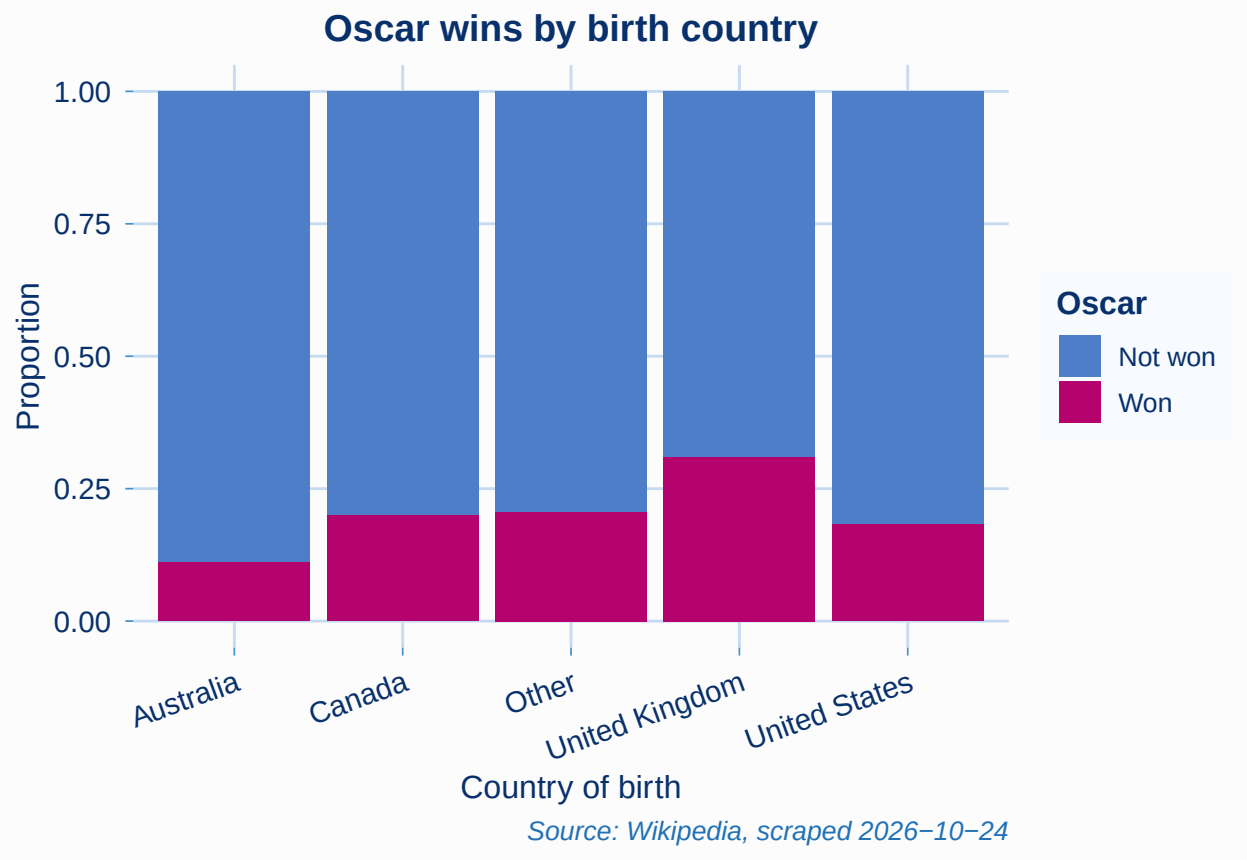
```
## $ birth_country <fct> United States, United States, Other, Other, United
 ↪  State~
## $ n_children    <int> 2, NA, 1, NA, NA, 5, NA, 5, 2, 2, 2, 1, 1, 1, 1, 3, 1,
 ↪  N~
## $ oscar_won     <chr> "0", "0", "0", "0", "0", "0", "0", "0", "0", "0", "0",
 ↪  "~
## $ birth_year    <int> 1897, 1890, 1974, 1898, 1905, 1909, 1911, 1920, 1953,
 ↪  19~
## $ inducted_year <int> 1960, 1960, 2017, 1960, 1984, 1960, 1960, 1975, 2004,
 ↪  19~
## $ age_at_star   <int> 63, NA, 43, NA, NA, 51, 48, 55, 51, 44, 43, 56, NA, 78,
 ↪  ~
```

Let's save the final table to a file named `actors_oscar.csv`.

```
output_path <- here::here("data", "clean", "actors_oscar.csv")
write_csv(actors_clean, output_path, na = "")
```

Visualization

```
actors_clean %>%
  drop_na(birth_country) %>%
  ggplot(aes(x = birth_country, fill = factor(oscar_won))) +
  geom_bar(position = "fill") +
  scale_fill_manual(values = c("0" = '#4E7EC7', "1" = '#B5036E'),
                    labels = c("Not won", "Won")) +
  labs(x = "Country of birth", y = "Proportion", fill = "Oscar",
       title = "Oscar wins by birth country",
       caption = "Source: Wikipedia, scraped 2026-10-24") +
  theme_myreport() +
  theme(axis.text.x = element_text(angle = 20, hjust = 1))
```



This chart shows that about 30% of the actors on this list who were born in the UK have won an Oscar. In comparison, about 20% of the actors on the list from Canada and the U.S. have won an Oscar. However, Australia has the lowest percentage of Oscar-winning actors: about 10%. It's worth noting, though, that there were significantly fewer people from the UK and Australia than from the U.S., so the data may not accurately reflect the actual situation.

List of references

Wikipedia contributors. (2026, October 24). List of actors with Hollywood Walk of Fame motion picture stars. In *Wikipedia*. Retrieved October 26, 2026, from https://en.wikipedia.org/wiki/List_of_actors_with_Hollywood_Walk_of_Fame_motion_picture_stars